

Manual de Arquitectura y Estructura del Proyecto

Agente IA (Orquestador)

3 de julio de 2026

Índice

1. Introducción	2
2. Directorios Principales	2
3. Archivos de Configuración y Scripts Raíz	3
4. Conclusión	4

1. Introducción

Este manual describe la estructura de directorios y archivos presentes en el espacio de trabajo de Electrónica. El entorno está diseñado bajo una arquitectura de tres capas, que separa la lógica probabilística (LLMs) de la ejecución determinista (scripts), y también contiene recursos para IoT, Diseño de Circuitos Integrados (EDA) y Redacción Académica.

2. Directorios Principales

- **.agent/**: Contiene las instrucciones del sistema para el agente de IA (4 archivos: `AGENT_FRAMEWORK.md`, `AGENT_INSTRUCTIONS.md`, `latex.md`, `python.md`), definiendo su marco operativo, restricciones de entorno y protocolos de actuación.
- **directives/**: Capa 1 de la arquitectura. Contiene manuales de operación (SOPs) en formato YAML (17 archivos). Definen *qué* debe hacerse (ej. `scrape_website.yaml`, `analizar_imagen.yaml`, `git_update.yaml`, `evaluar_examen_estudiante.yaml`, `evaluar_practica.yaml`, `elaborar_examen.yaml`, `elaborar_ejercicios.yaml`, `imagen_a_kicad.yaml`).
- **execution/**: Capa 3 de la arquitectura. Contiene scripts deterministas en Python con una sola responsabilidad (14 archivos: `env_diagnostic.py`, `scrape_single_site.py`, `analizar_imagen.py`, `evaluar_examen.py`, `generar_informe.py`, `generar_informe_imagen.py`, `alert_user.py`, `elaborar_examen.py`, `elaborar_ejercicios.py`, `generar_examen_latex.py`, `generar_ejercicios_latex.py`, `generar_kicad_llm.py`, `generar_kicad_sch.py`, `extraer_netlist_imagen.py`).
- **.tmp/**: Directorio temporal utilizado por la capa de orquestación para almacenar estados de ejecución (como `run_state.json`) y artefactos temporales de procesamiento.
- **docs/**: Documentación técnica y académica (21 subdirectorios: `CIRC_DISP_ELECT/`, `SMPS/`, `EDA/`, `PROTECTOR_120VAC/`, `ZBAR_PRACTICAS/`, `EASYEDA/`, `IMAGENES/`, `RAG/`, `vLLM/`, `OPENCODE/`, `MEDIDOR_ENERGIA/`, `PC_ASUS/`, `PC_PARA_IA/`, `SERVIDOR_POWEREDGE_R610/`, `SISTEMA_INTERNAC/`, `CIRC_PARA_RESP_RAPIDAS/`, `PROYECTO_MECATRONICA/`, `Ventilador_3_Velocidades/`, `KICAD/`, `AGENTE_IA/`, `MANUAL/`, entre otros).
- **cursos/**: Material didáctico, notas de cursos de Electrónica, apuntes de Laboratorio y Tesis (6 subdirectorios: `DISP_ELECTRONICOS/`, `INT_ELECTRONICA/`, `TESIS/`, `PLAN_ESTUDIOS/`, `LABORATORIO_I_FISICA/`, `LABORATORIO_II_FISICA/`). Contiene además subcarpetas `examen_estudiante/` e `informe_examen/` para el flujo de evaluación.
- **directives/rubricas/**: Rúbricas en formato YAML para la evaluación de prácticas de laboratorio, inyectadas como contexto en el prompt de Gemini.
- **chroma_db/**: Base de datos vectorial autogenerada, utilizada por el sistema RAG (Retrieval-Augmented Generation) para la memoria y búsqueda semántica del agente.

- `Agente_EDA/`: Recursos específicos para el agente EDA, como esquemas (schemas) de validación y pruebas para la generación de netlists/BOM hacia EasyEDA.

3. Archivos de Configuración y Scripts Raíz

- `AGENTS.md`: Reglas globales del proyecto, convenciones de código y estructura general del espacio de trabajo.
- `.env`: Plantilla de variables de entorno con credenciales comentadas (`GROQ_API_KEY`, `GITHUB_TOKEN`) como referencia para la configuración local.
- `.groq_api_key`: Archivo oculto con la clave API de Groq (excluido de git vía `.gitignore`), usado por `rag_system.py` y `agent_eda.py`.
- `.gitignore`: Exclusiones de seguimiento git: `chroma_db/`, entornos virtuales, `__pycache__`, archivos auxiliares LaTeX, clave API de Groq.
- `requirements.txt`: Dependencias para los scripts deterministas de ejecución: `psutil` y `PyYAML`.
- `rag_system.py`: Script del Chatbot RAG. Vectoriza documentos (TEX, MD, PDF) en ChromaDB y gestiona las respuestas usando el LLM vía Groq.
- `agent_eda.py`: Agente especializado en EDA. Extrae netlists y BOM de archivos LaTeX (circuitikz) para exportar hacia EasyEDA Standard.
- `execution/evaluar_examen.py`: Evalúa exámenes y prácticas de laboratorio usando LLM multimodal (Gemini/OpenRouter). Renderiza el PDF en imágenes y envía en bloque al modelo junto con la System Instruction.
- `execution/elaborar_examen.py`: Genera exámenes escritos completos mediante LLM multimodal, con preguntas, opciones múltiples y solución paso a paso.
- `execution/elaborar_ejercicios.py`: Genera ejercicios de electrónica con LLM, estructurados en JSON para su posterior formateo LaTeX.
- `execution/generar_examen_latex.py` / `generar_ejercicios_latex.py`: Convierten los JSON de examen/ejercicios en documentos LaTeX compilables.
- `execution/generar_informe.py`: Genera un informe académico profesional en LaTeX a partir del JSON de evaluación producido por `evaluar_examen.py`.
- `execution/analizar_imagen.py`: Analiza una o más imágenes (JPG/PNG) con LLM multimodal (Gemini/OpenRouter) y devuelve un análisis estructurado en JSON.
- `execution/generar_informe_imagen.py`: Genera un informe profesional en LaTeX a partir del JSON de análisis producido por `analizar_imagen.py`.
- `execution/generar_kicad_llm.py` / `generar_kicad_sch.py` / `extraer_netlist_imagen.py`: Flujo EDA para convertir imágenes de circuitos o descripciones a esquemas KiCad.

- `flujo_evaluar_examen.py`: Orquestador (Layer 2) que ejecuta el flujo completo de evaluación: analiza el PDF con el LLM, genera el informe LaTeX y notifica al usuario.
- `flujo_elaborar_examen.py`: Orquestador (Layer 2) para el flujo de elaboración de exámenes (generar contenido → convertir a LaTeX → notificar).
- `flujo_elaborar_ejercicios.py`: Orquestador (Layer 2) para el flujo de elaboración de ejercicios de electrónica.
- `flujo_analizar_imagen.py`: Orquestador (Layer 2) del flujo de análisis de imágenes: analiza con LLM, genera informe LaTeX, presenta resumen y notifica al usuario.
- `flujo_imagen_a_kicad.py`: Orquestador (Layer 2) que convierte imágenes de circuitos a esquemas KiCad usando LLM.
- `clean_latex.py`: Utilidad para eliminar archivos auxiliares y temporales de compilación de LaTeX (.aux, .log, .out, etc.).
- `fix_latex.py`: Script de corrección automática de errores comunes en archivos LaTeX generados por LLM.
- `md_to_pdf.py` y `merge_pdfs.py`: Utilidades para la conversión y manipulación de documentos PDF.
- `ren_archivos.py`: Script para el renombrado masivo de archivos eliminando cadenas específicas.
- `test_generator.py` y `test_regex.py`: Pruebas unitarias (sin dependencias externas) para el generador JSON de EasyEDA y expresiones regulares.
- `db_state.json`: Archivo que lleva el registro del estado y actualizaciones incrementales de la base de datos RAG vectorial.
- `opencode.json`: Archivo de configuración que enlaza las instrucciones del agente alojadas en `.agent/` (`{"instructions": [".agent/*.md"]}`).
- `README.md`: Documento de presentación del repositorio con instrucciones de instalación, uso y descripción general del proyecto.
- `docs/MANUAL/manual.tex`: El presente manual en formato LaTeX, que describe la arquitectura, estructura y componentes del espacio de trabajo.
- `git-update.sh` y `update_repo.sh`: Scripts de automatización en bash para sincronización y actualización del repositorio Git.

4. Conclusión

Esta estructura garantiza una alta modularidad. El agente IA actúa como "middleware" o capa de orquestación (Capa 2), leyendo intenciones, buscando las instrucciones en `directives/`, delegando el trabajo duro a `execution/` y documentando hallazgos en `chroma_db/` o en archivos `.tex`. Todo ello cumpliendo con las reglas de codificación y formato estipuladas.